

Sistema para Empresa de Empleabilidad

Plataforma web "JobConnect" — Consumo de servicios con Fetch API

Modalidad: Grupos de máximo 5 estudiantes · **Duración:** 3 sesiones (planificación, desarrollo y cierre)

Herramienta de apoyo: NotebookLM (investigación, video e infografía) · **Valor:** 20% del curso ·

Control de versiones: Git obligatorio

Conformación de equipos: El proyecto se desarrolla en **grupos de máximo 5 estudiantes**. Todos los integrantes deben participar activamente en las tres fases y contribuir al repositorio Git del equipo.

1. Descripción del proyecto

Una empresa de empleabilidad llamada **JobConnect** necesita una plataforma web (únicamente *frontend*) que le permita administrar la información que maneja diariamente: candidatos, vacantes, empresas clientes, postulaciones, entrevistas y tareas de sus reclutadores. Como equipo de desarrollo, deben construir el **panel de administración** de JobConnect consumiendo servicios de una API mediante `fetch`, implementando operaciones CRUD completas (GET, POST, PUT, PATCH y DELETE) y un sistema de autenticación con inicio de sesión.

El sistema debe permitir que un reclutador inicie sesión y gestione los **6 módulos** del negocio. Todo el consumo de datos se realiza contra la API pública **DummyJSON** (<https://dummyjson.com>), mapeando cada recurso a una entidad del dominio de empleabilidad.

2. Módulos del sistema (6 CRUDs)

#	Módulo del negocio	Recurso (DummyJSON)	Métodos HTTP a implementar
1	Candidatos	<code>/users</code>	GET · POST · PUT · PATCH · DELETE
2	Vacantes	<code>/products</code>	GET · POST · PUT · PATCH · DELETE

#	Módulo del negocio	Recurso (DummyJSON)	Métodos HTTP a implementar
3	Empresas clientes	<code>/carts</code>	GET · POST · PUT · DELETE
4	Postulaciones	<code>/posts</code>	GET · POST · PATCH · DELETE
5 *	Entrevistas / notas	<code>/comments</code>	GET · POST · PATCH · DELETE
6	Tareas del reclutador	<code>/todos</code>	GET · POST · PATCH · DELETE

Autenticación: Login contra `/auth/login` con token guardado en `localStorage`. Si no hay token, el sistema no debe permitir el acceso a los módulos.

Credenciales de prueba: usuario `emilys` · contraseña `emilypass`

3. Requerimientos funcionales

Describen *qué* debe hacer el sistema.

Código	Requerimiento funcional
RF-01	El sistema debe permitir a un usuario iniciar sesión con usuario y contraseña contra el endpoint <code>/auth/login</code> .
RF-02	El sistema debe almacenar el token de sesión y usarlo en las peticiones que lo requieran mediante el header <code>Authorization</code> .
RF-03	El sistema debe restringir el acceso a los módulos si el usuario no está autenticado (no hay token válido).
RF-04	El sistema debe permitir cerrar sesión, eliminando el token almacenado.
RF-05	El sistema debe listar (GET) los registros de cada uno de los 6 módulos.
RF-06	El sistema debe permitir crear (POST) nuevos registros en cada módulo.
RF-07	El sistema debe permitir editar registros existentes usando PUT (reemplazo total) y/o PATCH (actualización parcial) según corresponda al módulo.
RF-08	El sistema debe permitir eliminar (DELETE) registros de cada módulo.
RF-09	El sistema debe mostrar mensajes de retroalimentación al usuario ante operaciones exitosas o fallidas.
RF-10	El sistema debe permitir navegar entre los 6 módulos desde una interfaz principal (menú o panel).

4. Requerimientos no funcionales

Describen *cómo* debe comportarse el sistema (calidad, restricciones).

Código	Requerimiento no funcional
RNF-01	El sistema debe ser únicamente frontend, desarrollado con HTML, CSS y JavaScript (vanilla o framework a elección del equipo).
RNF-02	El código debe estar organizado en carpetas (por ejemplo <code>/services</code> , <code>/pages</code> , <code>/assets</code>) siguiendo una estructura clara.
RNF-03	Todo el consumo de datos debe realizarse con <code>fetch</code> y manejo asíncrono (<code>async/await</code>).

Código	Requerimiento no funcional
RNF-04	La interfaz debe ser intuitiva, responsiva y usable en distintos tamaños de pantalla.
RNF-05	El manejo de errores debe controlarse con <code>try/catch</code> y no romper la aplicación ante fallos de red.
RNF-06	El proyecto debe gestionarse con Git: repositorio remoto, uso de ramas, commits descriptivos y frecuentes por parte de todos los integrantes.
RNF-07	El código debe ser legible: nombres significativos, indentación consistente y comentarios donde aporten claridad.
RNF-08	Las credenciales y tokens no deben quedar expuestos de forma insegura en el código fuente.
RNF-09	El proyecto debe incluir un archivo <code>README.md</code> con la documentación de uso e instalación.

Nota importante: DummyJSON *simula* las operaciones POST/PUT/PATCH/DELETE (devuelve la respuesta pero no persiste los cambios en su servidor). Esto es ideal para practicar el consumo de servicios sin montar un backend propio.

5. Fases del proyecto

DÍA 1 · Planificación

Investigación del dominio con NotebookLM

El equipo utiliza **NotebookLM** como herramienta de investigación. Cargan como fuentes: la documentación de Fetch API (MDN), la guía de DummyJSON, artículos sobre procesos de reclutamiento y empleabilidad, y material sobre autenticación con tokens. Con esas fuentes consultan a NotebookLM para comprender el dominio del negocio y los aspectos técnicos (diferencia entre PUT y PATCH, uso de *headers* con token, `async/await`).

En esta fase también **crean el repositorio Git** del equipo, definen la estructura de carpetas inicial y la estrategia de ramas.

Entregable: documento de planificación con la descripción del sistema, la tabla de módulos y endpoints, los requerimientos, el flujo de autenticación y un resumen del dominio generado con apoyo de NotebookLM. Repositorio Git creado.

DÍA 2 · Desarrollo

Consultas inteligentes a NotebookLM

El equipo programa el login y los 6 CRUDs con `fetch`, trabajando de forma colaborativa mediante ramas de Git y realizando commits frecuentes. Cada vez que enfrenten un bloqueo o duda técnica, la resuelven mediante consultas precisas a NotebookLM (por ejemplo: «¿por qué mi POST no envía el body?», «¿cómo protejo una página sin token?», «¿por qué aparece un error CORS?»).

Entregable: la aplicación funcionando (login + los 6 módulos) reflejada en el repositorio Git, y una bitácora con las principales dudas resueltas con NotebookLM.

DÍA 3 · Cierre y entrega

Documentación, video e infografía con NotebookLM

El equipo documenta y presenta el resultado apoyándose en NotebookLM:

- **Video de presentación:** usando la función de *Video Overview* / *resúmenes* de NotebookLM, generan una presentación del proyecto que explique qué hace el sistema JobConnect, sus módulos y cómo funciona, complementándola mostrando la app en ejecución.
- **Infografía:** con apoyo de NotebookLM (cargando su propio código y documentación como fuente) elaboran una infografía que resuma la arquitectura, los 6 módulos, los métodos HTTP y el flujo de autenticación.

- **Documentación (README):** cargan el código en NotebookLM y le piden ayuda para redactar el `README.md`.

Entregable final: repositorio Git con el código completo (login + 6 CRUDs) organizado en carpetas · video de presentación · infografía · `README.md` · documento reflexivo de cómo NotebookLM apoyó cada fase.

6. Entregables finales

- Enlace al repositorio Git con historial de commits y ramas.
- Aplicación frontend funcional (login + 6 CRUDs con fetch).
- Video de presentación del proyecto (con apoyo de NotebookLM).
- Infografía del sistema (con apoyo de NotebookLM).
- Archivo `README.md` completo.
- Documento reflexivo sobre el uso de NotebookLM en cada fase.
- Bitácora de dudas técnicas resueltas con NotebookLM.

Documento de trabajo — Segundo Mini Proyecto (Frontend). El uso de NotebookLM es obligatorio como herramienta de investigación, consulta y apoyo en la elaboración del video y la infografía. La rúbrica de evaluación se entrega en documento aparte.